

OAuth 2.0 and the "Road to Hell"

Revisiting Eran Hammer's Damning of OAuth After a Decade of Practice (2012) - ¡APIcryphal! 02



In this second edition of "¡APIcryphal!", I retell the story of OAuth 2.0 and reflect on Eran Hammer and his declaration that the finished framework is **the road to hell**.

Occasionally, among the daily business-as-usual, moments emerge that change the course of entire industries. The API landscape is a space as molded by legend as any other. While their accuracy to actual events remains debated, these tales remain the cornerstones of hallway tracks and executive talking points. These are API's apocryphal stories - the ¡APIcryphal!

TL;DR:

OAuth 2.0 is the industry-standard protocol for authorization, but it has its problems. Developers embarking on an implementation journey *must* proceed cautiously regarding the inherent complexities and seeming contradictions.



Observance

Over a decade ago, Eran Hammer took a stand that reverberated through the tech community. As the lead author and specification editor for OAuth from 2007 to 2011, Hammer was more than just a contributor; he passionately led difficult, if not sometimes tedious, work to secure what was an increasingly important part of online interactions. By 2012, however, Hammer had become completely disillusioned with OAuth 2.0, a significant revision to the protocol.

The story of OAuth is a tale of two versions: the original OAuth 1.0, a protocol born out of necessity, and its successor, OAuth 2.0, which promised to solve OAuth 1.0's limitations. Hammer, deeply involved in OAuth 1.0, envisioned a simple, secure, and interoperable protocol. However, as the enterprise world began to weigh in, OAuth 2.0 morphed into something Hammer could no longer endorse.

In July 2012, Hammer's disillusionment culminated in a dramatic resignation. He withdrew his name from the OAuth 2.0 specification, leaving behind a project he had

shepherded for half a decade. In his blog post, [culled from the Internet Archive's Wayback Machine and archived as a gist](#), he declared that OAuth 2.0 was "**the road to hell**". "OAuth 2.0 is a bad protocol," he wrote. "WS-* bad." Hammer feared that OAuth 2.0's complexity and extensibility would lead to widespread, insecure implementations for subsequent decades and could no longer continue in good conscience.

"The resulting specification is a designed-by-committee patchwork of compromises that serves mostly the enterprise. To be accurate, it doesn't actually give the enterprise all of what they asked for directly, but it does provide for practically unlimited extensibility. It is this extensibility and required flexibility that destroyed the protocol. With very little effort, pretty much anything can be called OAuth 2.0 compliant."

What happens when a leading proponent of an essential part of APIs becomes the biggest critic? And with more than a decade of hindsight, was he right?

Interpretation

Hammer's concerns were not unfounded. He argued that OAuth 2.0's complexity would be its downfall, making it difficult for developers to implement securely. Specifically, Hammer's concerns could be grouped into four major areas:

- **Lack of Interoperability:** One of the original goals of OAuth was to provide a standard way for different systems to interoperate. However, Hammer felt that OAuth 2.0 had become too flexible, leading to a situation where two OAuth 2.0 implementations might not necessarily work together out of the box.
- **Security Concerns:** OAuth 1.0 had built-in cryptographic signatures to ensure data integrity and authenticity. OAuth 2.0 moved away from this, relying more on bearer tokens and HTTPS for security. Hammer believed this made the protocol less secure by default and placed more burden on developers to ensure safety.
- **Corporate Influence:** Hammer expressed concerns that large corporations had too much influence over the OAuth 2.0 specification, leading to decisions that favored these companies' interests over the broader community.
- **"Framework" over "Protocol":** Hammer was angered at the enterprise participant's insistence that OAuth 2.0 be called a "framework" rather than a "protocol". Rather

than providing a precise specification for which implementations would either cleanly pass or fail, OAuth 2.0 offered multiple ways to accomplish the same task. Hammer strongly believed this approach would lead to confusion and mistakes.

Given what we know today, how would we score these concerns?

Lack of Interoperability

As far as interoperability goes, this concern is extremely valid. OAuth shouldn't be thought of as a prescriptive, A->B->C, chain of how to do something. Instead, it is a collection of concepts and interactions ("roles", "tokens", "grant types", etc.) from which users can implement their authentication. And, as it turns out, people assemble those pieces in all sorts of varying ways. As Evert Pot points out in his article, [Does OAuth 2 Have a Usability Problem? \(Yes!\)](#) he says,

"-the popular [passport.js](#) project has 538(!) modules for authenticating with various services, most of which likely use OAuth2. All of these are NPM packages."

Despite OAuth2 being well defined, the full breadth of what is possible with the framework makes telling users, "We use OAuth2," not enough. Instead, as Evert lists, you have to include details such as:

- We use OAuth2.
- We use the authorization_code flow.
- Your client_id is X.
- Our 'token endpoint' is Y.
- Our 'authorization endpoint' is Z.
- We require PKCE.
- Requests to the "token" endpoint require credentials to be sent in a body.
- Any custom non-standard extensions.

That's a lot! New or time-strapped developers can't just pick up a "generic" OAuth2 implementation. To use OAuth successfully, developers must be familiar with all these terms. A consequence is that API vendors that use OAuth2 nearly always implement

custom SDKs to insulate users from these implementation details. The pain point is why products like [Nango](#) and [Passport.js](#) exist.

One point awarded to Hammer.

Security Concerns

A significant aspect of OAuth 1.0 was combining multiple pieces of information into a hash, which was then passed to the server. This scheme was meant to preserve the integrity of the request from meddling, primarily from "man-in-the-middle" attacks.

The OAuth 2.0 decision to rely on TLS rather than OAuth 1's signatures is interesting. In the late aughts and early 2010s far fewer sites were secured with HTTPS. The process of getting and configuring a certificate was expensive and time-consuming. From Hammer's perspective, ceding this vital security aspect to a protocol developers were loath to implement was worthy of criticism.

However, Hammer did not anticipate TLS practices would change industry-wide in the ensuing years. The push for "HTTPS everywhere", the browsers' presentation of secure vs. unsecured sites, and specific organizations like [Let's Encrypt](#) forced a sea change in practice. The result was significant changes for the better in how people obtain and apply certificates. In 2023, it is rare for any web-based request not to be done with TLS.

In this scenario, securing the message is best left out of OAuth and is done in the proper place.

Hammer - 1, OAuth 2.0 - 1

Corporate Influence

Hammer characterized the authors of OAuth as a scrappy band of developers who were trying to get things done. As those individuals moved on to other projects, Hammer was distressed as they were replaced by individuals representing large corporate interests. These interests often were from the perspective of companies with large technology teams, bigger budgets, and longer timelines than most.

What seems viable from that perspective is quite different from what can be easily supported by rapid prototype makers and startup tinkerers. Subsequently, as I cover in a recent piece on authentication types, it is why most initial API offerings eschew OAuth today and provide some form of [simple bearer authentication via API keys](#). It is less intimidating and far easier for any developer to just "try something" in this manner.

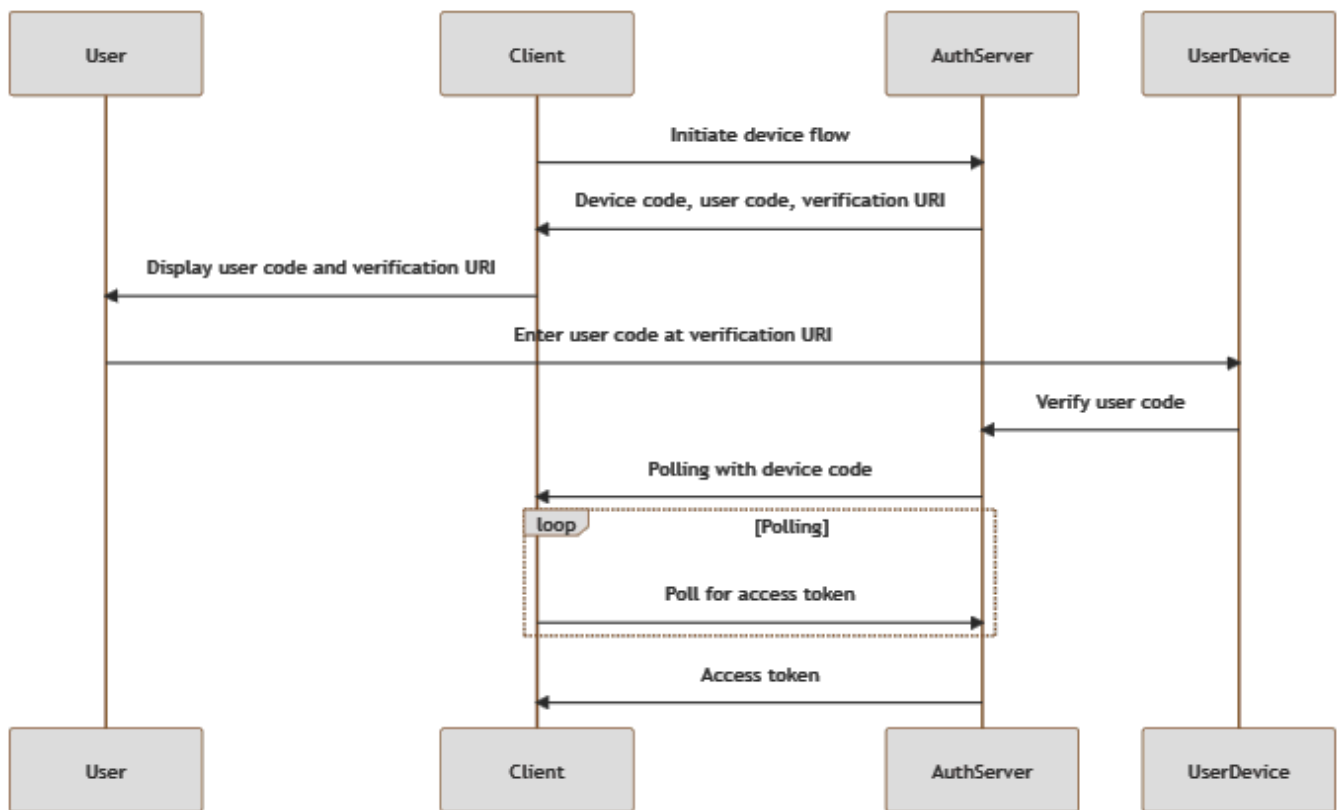
Context includes who you're solving for. Corporate needs are still valid but should not come at the expense of essential use cases.

Hammer - 2, OAuth 2.0 - 1

"Framework" over "Protocol"

A heavy implication of Eran's concerns is that a protocol only allows things to be done a single way. An implementation of a protocol either passes or fails the specification that defines the protocol - there's no gray area for interpretation.

However, by allowing for a significant amount of extensibility, OAuth 2.0 can support new flows unknown or unconsidered during the original framework's authoring. For example, take the "Device Authorization Flow", pictured below. It would have been difficult, if not impossible, to anticipate the explosion of streaming services and the need to approve account access across various "smart" entertainment devices in 2012 - devices that, subsequently, have poor user input mechanisms.



An OAuth Device Authorization Flow Sequence Diagram

When attempting to attach your account to any hotel TV or Roku device, you benefit from the flexibility of OAuth 2.0's framework. This ability to adapt to support new use cases will keep OAuth 2.0 as the primary authentication approach for some time.

Ladies and gentlemen, technologists of all ages - *we have a tie!*

Hammer - 2, OAuth 2.0 - 2

Settling the Score

All of Eran Hammer's specific concerns ultimately meant that developers who implemented OAuth 2.0 would deal with greater complexity - complexity that would undermine security, embolden malicious actors, and lead to high-profile, highly damaging breaches.

On the one hand, even in 2023, [we're still discovering misconfigured OAuth implementations that imperil millions of user accounts](#). In 2021, [Salt Security revealed critical API security vulnerabilities](#) in several sites' OAuth implementations, including Grammarly, Vidio, and Bukalapak. These vulnerabilities had the potential to

compromise user credentials and enable full account takeovers. To use OAuth 2.0 requires specialized understanding that is difficult to acquire from a handful of Google or Stack Overflow searches.

On the other hand, if one does that work, they'll realize that many of these exploits are related to the widely discouraged "Implicit Flow", an approach initially intended for single-page application (SPA) clients that cannot securely store a client secret. The Implicit Flow has been superseded by the Authorization Code Flow with Proof Key for Code Exchange (PKCE). What some might decry as additional complexity I cite as an example of how OAuth can evolve over time to meet new, emergent needs.

Every time I attempt to kick the tires on an API that requires OAuth 2.0, I inwardly groan because I know getting access will take me more than a few minutes. However, if I am building upon a dependency for the long term, I recognize that I want an API that exhibits the maturity of an OAuth implementation.

Using OAuth 2.0 is a journey. It may be long, but it doesn't have to be a one-way to hell. **Winner: OAuth 2.0.**

Keys

Today, OAuth 2.0 is ubiquitous, yet it's not without its issues. The protocol's flexibility has resulted in a fragmented landscape where interoperability is not guaranteed. The proliferation of SDKs and the necessity for detailed implementation instructions underscore the complexity that Hammer warned about.

To mitigate these issues, companies must prioritize OAuth education, invest in robust security practices, and consider using managed services that abstract away some of the protocol's complexities. It's also crucial to stay updated with the latest OAuth best practices and extensions, such as the Authorization Code Flow with PKCE, which enhances security for public clients.

Reversal

Eran Hammer was not alone in his criticisms of OAuth 2.0. Some of those vocal supporters went on to create alternatives to OAuth. Stevie Graham is the founder of Teller Inc, a service that "lets you securely connect to your financial accounts". To achieve this, [he created TAuth](#), an authentication protocol that is meant to address

OAuth 2.0's perceived shortcomings for the fintech space. A chief feature is OAuth 1.0-like signatures.

Other efforts are difficult to find, if nonexistent. For all the benefits TAuth might have claimed in securing APIs, it barely registers as more than an interesting footnote in the story of OAuth 2.0's dominance.

Legacy

After leaving the OAuth 2.0 steering committee, Eran Hammer continues to have a long and productive career. He was highly influential during his tenure with the retail giant WalMart. He also was a leading voice on the Hapi Node API framework. While it's clear that Hammer's decision, at the time, was painful and came from a place of authentic concern and deep passion, it does not define his legacy.

The "road to hell" declaration *was* a watershed moment, a cautionary tale about the complexities of standardization and the perils of compromise. While not entirely prophetic, his concerns influenced the ongoing conversation about API security. OAuth 2.0, for all its imperfections, shaped the security landscape of the web. It's a testament to the protocol's resilience and the industry's ability to adapt and iterate that has contributed to its longevity.

In the end, the road to this point may not have been as dire as Eran Hammer predicted. OAuth's journey is anything but a straight line, a road which weaves back and forth between innovation and security. Yet, for numerous reasons, it will be the one that all sorts of developers will continue to travel for the foreseeable future.